



Aprendizaje Autónomo 2

Técnicas Avanzadas de Testing de Software

Estudiante: Cristina Isabel Zambrano Cedeño

Tutor: Jonathan Eduardo Tito Ontaneda

Materia: Diseño de Pruebas, Control
de Calidad y Mantenimiento

JUNIO 2026

Diseño de Experimentos y Pruebas Combinatorias

1. Especificación del Software: Sistema bajo Prueba (SUT)

- **Nombre:** GlobalFly Booking Engine
- **Componente:** Módulo de validación de Tarifas y Rutas de Vuelo.
- **Objetivo:** Verificar que el sistema calcule correctamente la disponibilidad, restricciones de equipaje y precio base según las opciones seleccionadas por el usuario.

Factores y Niveles Seleccionados

Para cumplir con las condiciones del trabajo, definimos 5 factores. Tres de ellos que son Tipo_Pasajero, Destino y Aerolínea poseen 5 niveles cada uno, mientras que los otros dos, Clase_Cabina y Seguro_Viaje, poseen 2 y 3 niveles respectivamente.

Factor (Variable)	Tipo	Niveles (Valores Posibles)	Cantidad de niveles
1.Tipo_Pasajero	Requisito >4	Adulto, Niño, Bebé, Senior (Tercera_Edad), Militar/Diplomático	5
2. Destino	Requisito >4	Norteamérica, Europa, Asia, Sudamérica, África	5
3. Aerolínea	Requisito >4	Oneworld, SkyTeam, Star Alliance, Low-Cost A, Low-Cost B	5
4. Clase_Cabina	Estándar	Turista, Turista Premium, Business	3
5. Seguro_Viaje	Estándar	Incluido, No Incluido	2

Si hiciéramos una prueba exhaustiva convencional (5x5x5x3x2), tendríamos que ejecutar 750 casos de prueba. Al aplicar DoE (*Pairwise*), buscaremos reducir este número a menos del 5%.

Herramienta DoE Seleccionada

- **Herramienta: PICT (Pairwise Independent Combinatorial Testing)** desarrollada por Microsoft.
- La elección de esta herramienta se debe a que es un código abierto e industrial estándar para pruebas combinatorias. Utiliza un algoritmo codicioso (*greedy*) de optimización que garantiza que cada par de niveles de cualquier factor se pruebe de forma conjunta al menos una vez, maximizando la detección de defectos de interacción con el mínimo número de ejecuciones.

Archivo de Entrada para la Herramienta (modelo.txt)

Para que PICT procese las variables, se crea un archivo de texto plano estructurando los factores y sus niveles separados por comas:

```
Tipo_Pasajero: Adulto, Nino, Bebe, Senior, Militar
Destino:       Norteamerica, Europa, Asia, Sudamerica, Africa
Aerolinea:     Oneworld, SkyTeam, StarAlliance, LowCostA,
LowCostB
Clase_Cabina:  Turista, TuristaPremium, Business
Seguro_Viaje:  Incluido, No Incluido
```

Informe Generado por la Herramienta (Output de PICT)

Al ejecutar la herramienta mediante la consola de comandos, el algoritmo de diseño de experimentos analiza las matrices combinatorias y reduce los 750 casos iniciales a **solo 27 casos de prueba óptimos**.

A continuación se muestra el reporte de combinaciones estratégicas arrojado por el motor matemático:

Tipo_Pasajero	Destino	Aerolinea	Clase_Cabina	Seguro_Viaje
Adulto	Europa	Oneworld	Turista	Incluido
Senior	Norteamerica	LowCostA	TuristaPremium	Incluido
Nino	Asia	SkyTeam	Business	No_Incluido
Militar	Sudamerica	StarAlliance	Turista	No_Incluido
Bebe	Africa	LowCostB	Business	Incluido
Adulto	Asia	LowCostA	Business	No_Incluido
Senior	Sudamerica	LowCostB	Turista	No_Incluido
Militar	Norteamerica	SkyTeam	TuristaPremium	Incluido
Bebe	Europa	StarAlliance	TuristaPremium	No_Incluido
Nino	Sudamerica	Oneworld	TuristaPremium	Incluido
Adulto	Norteamerica	StarAlliance	Business	Incluido
Militar	Africa	Oneworld	Business	No_Incluido
Bebe	Asia	Oneworld	Turista	No_Incluido
Senior	Europa	SkyTeam	Business	Incluido
Nino	Norteamerica	LowCostB	Turista	No_Incluido
Adulto	Africa	SkyTeam	TuristaPremium	No_Incluido
Militar	Europa	LowCostB	TuristaPremium	No_Incluido
Nino	Europa	LowCostA	Business	Incluido
Senior	Africa	StarAlliance	Turista	Incluido
Bebe	Sudamerica	SkyTeam	Turista	Incluido
Adulto	Sudamerica	LowCostB	TuristaPremium	Incluido
Militar	Asia	LowCostA	Turista	Incluido
Bebe	Norteamerica	Oneworld	Business	No_Incluido

Senior	Asia	StarAlliance	TuristaPremium	No_Incluido
Nino	Africa	Oneworld	TuristaPremium	No_Incluido
Nino	Europa	StarAlliance	Turista	No_Incluido
Adulto	Norteamerica	LowCostB	Business	No_Incluido

Matriz de Casos de Prueba Desarrollados

A partir de los resultados matemáticos del DoE, el analista de control de calidad (QA) redacta y formaliza la suite de pruebas. Aquí se presentan los primeros 10 casos de prueba listos para su ejecución o automatización en herramientas de gestión como Jira/Xray:

ID Caso	Factor 1: Pasajero	Factor 2: Destino	Factor 3: Alianza/ Línea	Factor 4: Cabina	Factor 5: Seguro	Resultado Esperado
CP-01	Adulto	Europa	Oneworld	Turista	Incluido	Tarifa regular calculada con políticas de equipaje transatlántico y cobertura médica activada.
CP-02	Senior	Norteamérica	LowCostA	Turista Premium	Incluido	Aplicación de descuento de tercera edad sobre tarifa de bajo costo y seguro básico.

ID Caso	Factor 1: Pasajero	Factor 2: Destino	Factor 3: Alianza/ Línea	Factor 4: Cabina	Factor 5: Seguro	Resultado Esperado
CP-03	Niño	Asia	SkyTeam	Business	No Incluido	Cálculo de tarifa infantil en cabina ejecutiva; requiere validación de acompañante adulto.
CP-04	Militar	Sudamérica	StarAlliance	Turista	No Incluido	Exención de tasas e impuestos gubernamentales aplicada por estatus militar en ruta sur.
CP-05	Bebé	África	LowCostB	Business	Incluido	Emisión de boleto sin asiento asignado (en regazo) en cabina preferencial con seguro especial.

ID Caso	Factor 1: Pasajero	Factor 2: Destino	Factor 3: Alianza/ Línea	Factor 4: Cabina	Factor 5: Seguro	Resultado Esperado
CP-06	Adulto	Asia	LowCostA	Business	No Incluido	Alerta del sistema: La aerolínea de bajo costo seleccionada no posee cabina Business para Asia.
CP-07	Senior	Sudamérica	LowCostB	Turista	No Incluido	Tarifa económica limpia recalculada con penalización por equipaje de bodega.
CP-08	Militar	Norteamérica	SkyTeam	Turista Premium	Incluido	Verificación de acceso prioritario a cabina intermedia con protección de viaje activa.

ID Caso	Factor 1: Pasajero	Factor 2: Destino	Factor 3: Alianza/ Línea	Factor 4: Cabina	Factor 5: Seguro	Resultado Esperado
CP-09	Bebé	Europa	StarAlliance	Turista Premium	No Incluido	Validación de cuna disponible a bordo en trayecto largo para tarifa de infante.
CP-10	Niño	Sudamérica	Oneworld	Turista Premium	Incluido	Descuento menor de edad aplicado sobre clase intermedia y póliza de menores.

Al aplicar la técnica combinatoria *Pairwise* mediante la herramienta PICT, se logró una reducción del 96.4% en el esfuerzo de pruebas (pasando de 750 combinaciones teóricas a solo 27 escenarios físicos).

Científicamente, la literatura de ingeniería de software demuestra que la mayoría de los defectos críticos son causados por fallas en un solo factor o por la interacción anómala de dos factores combinados. Esta especificación garantiza la cobertura total de todas las interacciones de pares posibles, ofreciendo el estándar más alto de calidad con el menor costo operativo.

Análisis Estático del Código

Para esta parte se seleccionó la herramienta Coverage.py para entornos Python. Esta herramienta proporciona un análisis de cobertura de ramas (Branch Coverage). Esto significa que no solo mide si una línea con un condicional (if, while) se ejecutó, sino que evalúa de forma binaria si el flujo del programa tomó tanto el camino del Verdadero (True) como el del Falso (False) para cada decisión del algoritmo.

A continuación, se presenta la implementación clásica del algoritmo de búsqueda binaria sobre un arreglo ordenado.

```
# busqueda_binaria.py

def buscar_binario(lista, objetivo):
    izquierda = 0
    derecha = len(lista) - 1

    while izquierda <= derecha:
        medio = (izquierda + derecha) // 2

        if lista[medio] == objetivo:
            return medio # Elemento encontrado (Decisión 1 - Rama True)
        elif lista[medio] < objetivo:
            izquierda = medio + 1 # Buscar en la mitad derecha (Decisión
2 - Rama True)
        else:
            derecha = medio - 1 # Buscar en la mitad izquierda (Decisión
2 - Rama False)
    return -1 # Elemento no encontrado (Bucle termina - Rama False)
```

También se diseñó un archivo de pruebas inicial con pocos casos para observar cómo se comporta la herramienta cuando la cobertura es deficiente o incompleta.

```
# test_inicial.py
from busqueda_binaria import buscar_binario

def ejecutar_pruebas_iniciales():
    mi_lista = [10, 20, 30, 40, 50]

    # Caso 1: El elemento está exactamente en el medio
    assert buscar_binario(mi_lista, 30) == 2

    print("¡Pruebas iniciales ejecutadas con éxito!")

if __name__ == "__main__":
    ejecutar_pruebas_iniciales()
```

Para medir las ramas ejecutadas con Coverage.py, corremos los siguientes comandos en la terminal:

Python -m coverage report -m

Name	Stmts	Miss	Branch	BrPart	Cover	Missing
busqueda_binaria.py	11	4	6	2	53%	12-17
test_inicial.py	7	0	2	1	89%	12->exit
TOTAL	18	4	8	3	65%	

Como podemos observar, busqueda_binaria.py obtuvo solo un 53% de cobertura. La columna BrPart (Ramas Parciales) indica que se detectaron 2 decisiones que quedaron a medias: el bucle while nunca se evaluó como Falso ya que nunca terminó sin encontrar el elemento y el condicional if jamás se ejecutó.

Para forzar al algoritmo a recorrer todas las decisiones y ramas lógicas tanto el True como el False de cada if y while, se expande el suite de pruebas en un nuevo archivo con el siguiente código:

```
# test_avanzado.py
from busqueda_binaria import buscar_binario

def ejecutar_pruebas_avanzadas():
    mi_lista = [10, 20, 30, 40, 50]

    # CP-01: Elemento en el medio (Decisión 1 -> True)
    assert buscar_binario(mi_lista, 30) == 2

    # CP-02: Elemento en la mitad derecha (Decisión 1 -> False, Decisión
    2 -> True)
    assert buscar_binario(mi_lista, 50) == 4

    # CP-03: Elemento en la mitad izquierda (Decisión 1 -> False,
    Decisión 2 -> False)
    assert buscar_binario(mi_lista, 10) == 0

    # CP-04: Elemento inexistente / Menor (Bucle while -> Termina en
    False, retorna -1)
    assert buscar_binario(mi_lista, 5) == -1

    # CP-05: Elemento inexistente / Mayor (Prueba de límites vacíos)
    assert buscar_binario([], 10) == -1

    print("¡Todos los caminos lógicos han sido cubiertos con éxito!")

if __name__ == "__main__":
    ejecutar_pruebas_avanzadas()
```

Ejecución y resultado de la cobertura mejorada:

```
D:\UNI\Sem May- Sep\Diseño de Pruebas, Control de Calidad y Mantenimiento>python -m coverage report -m
```

Name	Stmts	Miss	Branch	BrPart	Cover	Missing
busqueda_binaria.py	11	0	6	0	100%	
test_avanzado.py	11	0	2	1	92%	24->exit
TOTAL	22	0	8	1	97%	

Impresión Gráfica de Cobertura (HTML Report)

Coverage.py permite exportar un reporte interactivo en HTML que resalta visualmente las líneas y decisiones tomadas. Al ejecutar coverage html, se genera una interfaz en el navegador web con la siguiente estructura visual:

Coverage for test_avanzado.py: 92% X

11 statements 11 run 0 missing 0 excluded 1 partial

« prev ^ index » next coverage.py v7.14.1, created at 2026-06-03 18:22 -0500

```
1 # test_avanzado.py
2 from busqueda_binaria import buscar_binario
3
4 def ejecutar_pruebas_avanzadas():
5     mi_lista = [10, 20, 30, 40, 50]
6
7     # CP-01: Elemento en el medio (Decisión 1 -> True)
8     assert buscar_binario(mi_lista, 30) == 2
9
10    # CP-02: Elemento en la mitad derecha (Decisión 1 -> False, Decisión 2 -> True)
11    assert buscar_binario(mi_lista, 50) == 4
12
13    # CP-03: Elemento en la mitad izquierda (Decisión 1 -> False, Decisión 2 -> False)
14    assert buscar_binario(mi_lista, 10) == 0
15
16    # CP-04: Elemento inexistente / Menor (Bucle while -> Termina en False, retorna -1)
17    assert buscar_binario(mi_lista, 5) == -1
18
19    # CP-05: Elemento inexistente / Mayor (Prueba de límites vacíos)
20    assert buscar_binario([], 10) == -1
21
22    print("¡Todos los caminos lógicos han sido cubiertos con éxito!")
23
24 if __name__ == "__main__":
25     ejecutar_pruebas_avanzadas()
```

24 ->exit

« prev ^ index » next coverage.py v7.14.1, created at 2026-06-03 18:22 -0500

Evaluación sobre la Utilidad de la Herramienta

El uso de herramientas de análisis de cobertura de decisión como Coverage.py aporta un valor crítico indispensable en los ciclos de calidad de software. Hay tres puntos en los que es de gran ayuda.

En la Visibilidad de Puntos Ciegos es de gran ayuda ya que nos permite identificar de inmediato condiciones lógicas complejas o ramificaciones de control de excepciones (try/except, else ocultos) que los desarrolladores suelen pasar por alto en pruebas felices conocidas como happy path.

En cuanto a la Optimización del Esfuerzo que considero uno de los puntos vitales del uso de herramientas informáticas, evita que el equipo escriba cientos de casos de prueba redundantes. En su lugar, el reporte orienta al programador exactamente hacia el escenario de entrada matemático que falta para "destrabar" la rama que está en 0%.

Y finalmente en la Prevención de Regresiones al integrarse en canalizaciones de Integración Continua (CI/CD), actúa como una compuerta de calidad automatizada que rechaza código nuevo si este no cumple con un umbral de cobertura estructural preestablecido, ej. mínimo 85% de ramas.